

แอปโปรเจกต์ฟัง Frontend

เรียนเฉพาะฝั่งหน้าจอ (Frontend): จัดการข้อมูลแบบ API-ready และสร้างระบบ Login ให้ดูมืออาชีพ โดยไม่ต้องเขียน Backend เลย

fetch()

async/await

Promise

Form Validation

Session Storage

Route Guard

0 สารบัญ

PART 1

Mock Data → API (Frontend)

แยก Data Layer · จัดการ

loading/error/success · fetch จริงหรือจำลอง

PART 2

Login แบบ Professional (Frontend)

Validation · UI States · Session · Route Guard · Demo ลองกดได้จริง

1 PART 1: จาก Mock Data → API (ฟัง Frontend)

1.1 มุมมองของ Frontend



หลักคิดสำคัญ: ฟัง Frontend ไม่ต้องรู้ว่าข้อมูลมาจากไฟล์ JSON ในเครื่อง หรือจาก Server ที่อยู่ไหน
หน้าที่ของเราคือ "ส่งคำขอ (Request) แล้วรอผลตอบกลับ (Response) มาแสดงผล" การเขียนให้เป็น

ระเบียบจะทำให้ทีหลังเปลี่ยนแหล่งข้อมูลได้โดยไม่ต้องแก้หน้า HTML เลย

1.2 ขั้นตอนที่ 1 – แยก Data Layer ออกเป็นไฟล์ `api.js`

admin/api.js – สร้างไฟล์ใหม่ไว้จัดการข้อมูลทั้งหมด (Separation of Concerns)

```
// ไฟล์นี้มีหน้าที่เดียว: รับส่งข้อมูลกับแหล่งข้อมูล (Data Layer)
// โค้ดหน้า UI จะเรียกใช้แค่ฟังก์ชันพวกนี้ ไม่ต้องรู้ที่มาของข้อมูล

const API = {

  // เรียกข้อมูลลูกบ้านทั้งหมด
  async getUsers() {
    const res = await fetch("./dataTest.json");
    if (!res.ok) throw new Error(`HTTP error: ${res.status}`);
    return res.json();
  },

  // เรียกข้อมูลลูกบ้านคนเดียว
  async getUser(id) {
    const res = await fetch(`./dataTest.json`);
    if (!res.ok) throw new Error(`HTTP error: ${res.status}`);
    const users = await res.json();
    return users.find(u => u.id === Number(id));
  }
};
```

💡 **ประโยชน์:** ต่อไปถ้ามี Server จริง แค่เปลี่ยนบัสรัด URL ในไฟล์ `api.js` ที่เดียว ส่วน `script.js` (การแสดงผล) ไม่ต้องแตะเลย

1.3 ขั้นตอนที่ 2 – จำลอง API ด้วย Promise (เมื่อยังไม่มี Server)

admin/api.js – จำลองความล่าช้าของเครือข่าย (ดูเหมือน API จริง)

```
const API = {

  async getUsers() {
```

```

    // จำลองว่า Server ใช้เวลาดอบ 1.5 วิ (ตอน dev จะได้เห็น loading state)
    await new Promise(resolve => setTimeout(resolve, 1500));
    const res = await fetch("./dataTest.json");
    if (!res.ok) throw new Error(`HTTP error: ${res.status}`);
    return res.json();
  },

  // จำลอง error (เพื่อทดสอบหน้าแสดงข้อผิดพลาด)
  async getUsersFail() {
    await new Promise(resolve => setTimeout(resolve, 1500));
    throw new Error("Network error"); // ปลอมให้ฝั่ง caller จัดการ
  }
};

```

1.4 ขั้นตอนที่ 3 – ใช้ Async/Await + try/catch ในหน้าแสดงผล

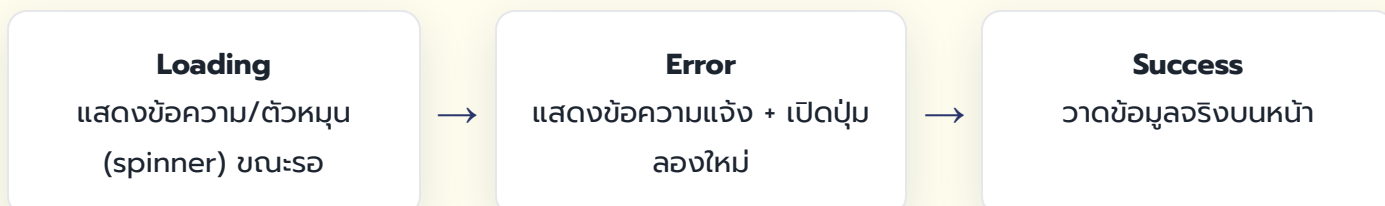
admin/script.js – ดึงข้อมูลผ่าน Data Layer (โค้ดสะอาดกว่าเดิม)

```

async function load() {
  setLoading(); // 1. แสดงสถานะ "กำลังโหลด"
  try {
    mainData = await API.getUsers(); // 2. ขอข้อมูล
    renderUserList(mainData); // 3. วาดข้อมูลลงหน้า
  } catch (err) {
    showError(err.message); // 4. มีปัญหา → แสดง error
  }
}

```

1.5 ขั้นตอนที่ 4 – จัดการ 3 สถานะ: Loading / Error / Success



admin/script.js – ฟังก์ชันจัดการสถานะ

```
function setLoading() {
  UserData.innerHTML = `

⌚ Loading data...</p>`;
}

function showError(msg) {
  UserData.innerHTML = `

❌ ${msg} - <button onclick="load()">ลองใหม่</button></p>`;
}


```

1.6 เทคนิคขั้นสูง – ยกเลิก request ที่ไม่ต้องการ (AbortController)

admin/api.js – ยกเลิก fetch ถ้าผู้ใช้เปลี่ยนหน้าเร็วเกินไป

```
let currentAbort = null;

async getUsers() {
  // ยกเลิก request ก่อนหน้า (ถ้ายังไม่เสร็จ) กันข้อมูลมาซ้อนกัน
  if (currentAbort) currentAbort.abort();
  currentAbort = new AbortController();

  const res = await fetch("./dataTest.json", { signal: currentAbort.signal });
  if (!res.ok) throw new Error(`HTTP error: ${res.status}`);
  return res.json();
}
```

1.7 สรุปเส้นทางข้อมูลใหม่

ชั้น (Layer)	ไฟล์	หน้าที่
Data Layer	api.js (ใหม่)	จัดการ fetch / จำลอง API / ตรวจสอบ error
Logic Layer	script.js	เรียกใช้ api.js และวาดผล (render)
View Layer	admin.html	โครงหน้าเว็บ ไม่ยุ่งกับข้อมูล

2.1 มองจากฝั่ง Frontend — ปัญหาของ login.js เดิม

⚠️ ปัญหาที่เห็นได้ชัดจากฝั่ง Frontend:

- รหัสผ่าน `admin/123` เปิดในโค้ดใครก็เห็น — อย่างน้อยควร "ปิด" การทำงานนี้ไว้เบื้องหลัง
- กดปุ่ม Enter หรือ Submit เข้าได้เรื่อย ๆ → ส่ง request ซ้ำ
- ไม่มี Validation — กรอกอะไรก็ส่งไปหมด
- ไม่มีสถานะ "กำลังตรวจสอบ" — ผู้ใช้ไม่รู้วาระบบทำงานอยู่หรือค้าง
- หน้า Admin กันแค่ `sessionStorage` ที่แก้กันง่าย

Frontend ทำได้เต็มที่ 3 อย่าง: **UX (หน้าตา/การแจ้งเตือน)**, **Validation (ตรวจสอบความถูกต้อง)**, และ **Route Guard (ป้องกันหน้า)**

2.2 Client-side Validation — เช็คก่อนส่ง

login/login.js — ตรวจสอบกรอกครบ + รูปแบบถูกต้องก่อนส่ง

```
function validate(username, password) {
  if (!username.trim()) return "กรุณากรอกชื่อผู้ใช้";
  if (!password) return "กรุณากรอกรหัสผ่าน";
  if (password.length < 3) return "รหัสผ่านสั้นเกินไป";
  return null; // ผ่านทั้งหมด → คืนค่า null (ไม่มีข้อผิดพลาด)
}

// ใช้ใน submit handler
const err = validate(inputUsername, inputPassword);
if (err) {
  showError(err);
  return; // หยุด ไม่ส่ง request
}
```

💡 **หมายเหตุ:** Validation ฝั่ง Frontend เป็นแค่ "ชั้นแรก" เพื่อ UX ที่ดี ปกติจริงๆ ต้องตรวจสอบซ้ำที่ Server ด้วย — แต่สำหรับเรียนจบ Frontend คือสิ่งที่เราทำได้

2.3 UI States — Loading / Error / Success

login/login.js — กันกดซ้ำ + แสดงสถานะทุกช่วง

```
loginForm.addEventListener("submit", async (event) => {
    event.preventDefault();

    // ---- Loading state: ปิดปุ่มกันกดซ้ำ + เปลี่ยนข้อความ
    submitBtn.disabled = true;
    submitBtn.textContent = "⌚ กำลังตรวจสอบ ...";
    hideError();

    try {
        // จำลองการตรวจสอบ (Delay) — ค่อยส่งไป Server ตรงนี้
        await new Promise(resolve => setTimeout(resolve, 1000));

        if (inputUsername === "admin" && inputPassword === "123") {
            // ---- Success: ออกบัตรผ่าน แล้วไปหน้า Admin
            sessionStorage.setItem("isLoggedIn", "true");
            window.location.href = "../admin/admin.html";
        } else {
            // ---- Error: แจ้งให้รู้
            showError("ชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง");
        }
    } catch (err) {
        showError("เกิดข้อผิดพลาด กรุณาลองใหม่");
    } finally {
        // คืนปุ่มเป็นปกติเสมอ (ทั้งสำเร็จและพลาด)
        submitBtn.disabled = false;
        submitBtn.textContent = "เข้าสู่ระบบ";
    }
});
```

2.4 ปุ่มสลับดูรหัสผ่าน + autocomplete

login/login.html — ฟีเจอร์ UX เล็ก ๆ ที่ทำให้ดูมืออาชีพ

```
<!-- ช่องกรอกรหัสผ่าน + ปุ่มสลับดู -->
<div class="password-wrap">
```

```
<input type="password" id="password" autocomplete="current-password">
<button type="button" id="togglePass">👁️</button>
</div>
```

login/login.js – ทำงานสลับ show/hide

```
togglePass.addEventListener("click", () => {
  const isHidden = passwordInput.type === "password";
  passwordInput.type = isHidden ? "text" : "password";
  togglePass.textContent = isHidden ? "👁️" : "🔒";
});
```

2.5 จัดการ Session อย่างถูกวิธี

ที่เก็บ	อยู่ได้นานแค่ไหน	เหมาะกับ
<code>sessionStorage</code>	ปิดแท็บ = หาย	บัตรผ่าน "ชั่วคราว" ของระบบนี้ ✅
<code>localStorage</code>	อยู่จนกว่าจะลบเอง	ของที่จำข้ามครั้ง เช่น ธีม, token
ตัวแปรใน JS	รีเฟรชหน้า = หาย	ค่าชั่วคราวที่ไม่ต้องเก็บ

💡 **ปกติระบบ Login ควรเก็บ "token" ที่ Server ออกให้** แต่ถ้าขอบเขตเรียนแค่ Frontend วิธีที่ยอมรับได้คือ เก็บค่าสถานะใน `sessionStorage` แบบที่โปรเจกต์ทำ และปรับให้สมบูรณ์ตามข้อ 2.6–2.8

2.6 Route Guard ฝั่งหน้าเว็บ – กันเปิดหน้า Admin ตรง ๆ

admin/script.js – บรรทัดแรกของไฟล์ (เดิมมีแล้ว ทำให้แน่นขึ้น)

```
// 1. ยังไม่ล็อกอิน → ดึงกลับหน้า Login
if (sessionStorage.getItem("isLoggedIn") !== "true") {
  // จำว่าตอนแรกอยากเข้าหน้าไหน (ต่อไปจะได้กลับมาที่เดิมได้)
  sessionStorage.setItem("redirectAfterLogin", location.href);
  window.location.href = "../login/login.html";
}
```

```
throw new Error("No permission"); // หยุดรันโค้ดที่เหลือทันที
}
```

2.7 Auto-logout เมื่อไม่ขยับนาน (Session Timeout)

admin/script.js – กันคนลืมล็อกเอาต์ทิ้งเครื่อง

```
// หมดเวลา 5 นาที → ออกจากระบบอัตโนมัติ
const TIMEOUT = 5 * 60 * 1000;
let timer;

function resetTimer() {
  clearTimeout(timer);
  timer = setTimeout(logout, TIMEOUT); // ไม่ขยับครบเวลา → logout
}

// ทุกครั้งที่ผู้ใช้ขยับเมาส์/พิมพ์ → นับเวลาดังแต่ต้น
['mousemove', 'keydown', 'click'].forEach(evt =>
  document.addEventListener(evt, resetTimer)
);
resetTimer();
```

2.8 Logout ที่ล้างข้อมูลครบถ้วน

admin/script.js – ล้างทุกอย่างที่เกี่ยวข้องกับ session

```
function logout() {
  sessionStorage.removeItem("isLoggedIn"); // ลบบัตรผ่าน
  sessionStorage.removeItem("redirectAfterLogin"); // ลบที่อยู่ที่เราจะไป
  window.location.href = "../index.html"; // กลับหน้าแรก
}
```

2.9 ลองเล่นจริง (Working Demo)

 **ลองกรอก:** admin / 123 = เข้าสำเร็จ · กรอกผิด = แสดง error · ปุ่มสลับดูรหัสผ่าน และสังเกตว่า

Login Demo (Frontend)

ชื่อผู้ใช้

admin

รหัสผ่าน

....



เข้าสู่ระบบ

2.10 Checklist ฟังก์ Frontend



มี Validation ก่อนส่ง (กรอกครบ + รูปแบบถูก)



มีสถานะ Loading (ปิดปุ่ม + เปลี่ยนข้อความ)



แสดง Error แบบอ่านเข้าใจง่าย (ภาษาไทย)



มีปุ่มสลับดูรหัสผ่าน + autocomplete ถูกต้อง



เก็บ session แบบชั่วคราว (sessionStorage)



Route Guard ป้องกันหน้า Admin เปิดตรงๆ



Auto-logout เมื่อไม่ขยับนาน



Logout ล้างข้อมูล session ครบถ้วน

